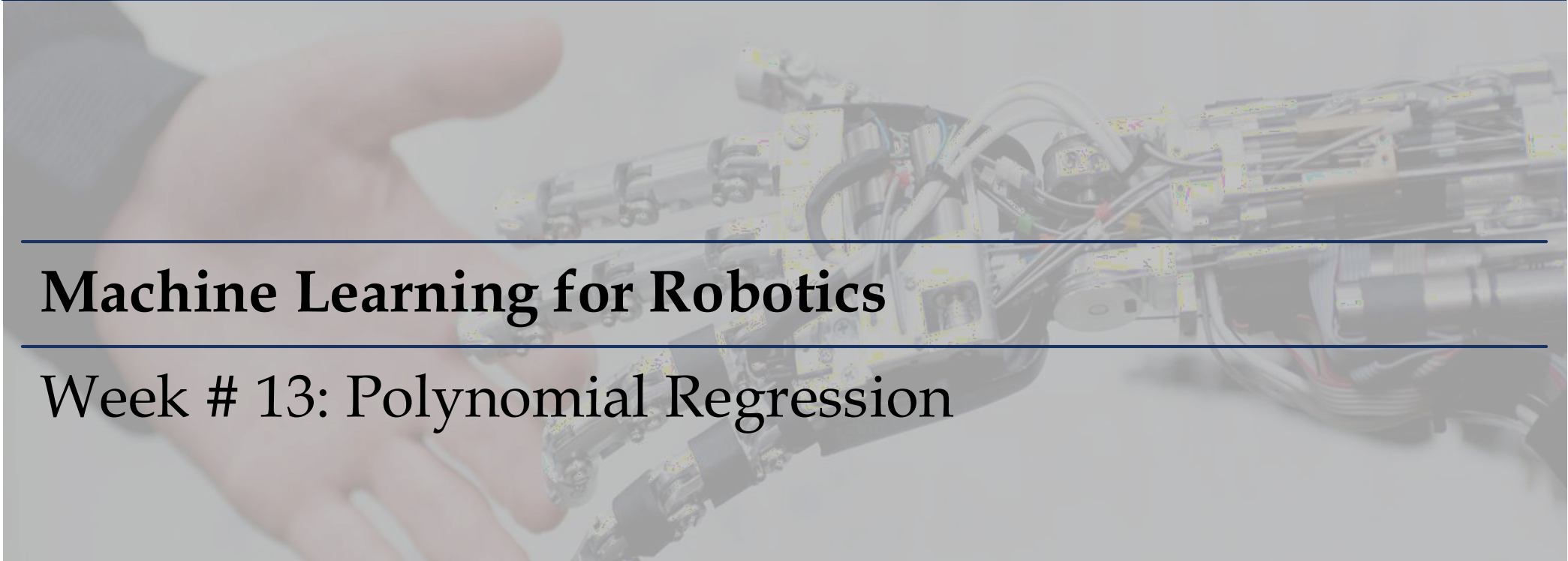


A background image showing a human hand holding a robotic arm. The robotic arm is covered with numerous small yellow rectangular markers, likely representing sensor data or feature points used in machine learning. The image is semi-transparent, allowing the text to be overlaid.

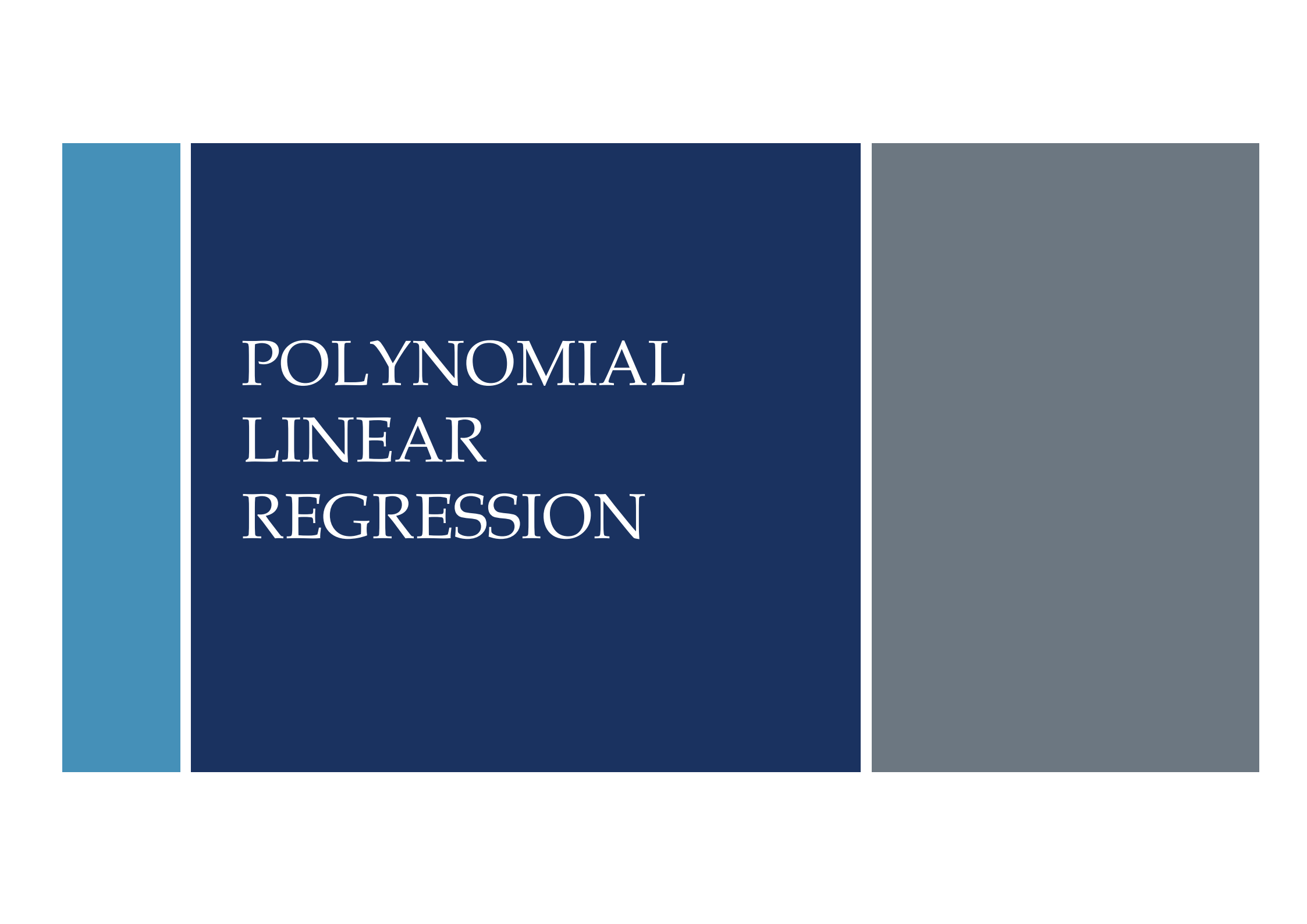
# Machine Learning for Robotics

## Week # 13: Polynomial Regression

**Dr. Maryam Iqbal**

## ABOUT INSTRUCTOR

- PhD EE (Controls, Automation & Rehabilitation Robotics)
- Research areas: Robotics & Automation, Artificial Intelligence, Machine Learning, Biomechanics, AI in Healthcare
- In charge : BSRIS Program at Bahria University
- Research Publications, Funded Projects and Patents
- Member of AI Task Force (Ministry of Planning, Development & Special Initiatives of Pakistan)
- Senior Member IEEE (Region 10) | General Secretary IEEE RAS (Region 3) | Member IEEE CSS
- Email: [maryamiqbal.buic@bahria.edu.pk](mailto:maryamiqbal.buic@bahria.edu.pk)
- LinkedIn: [linkedin.com/in/maryam-iqbal-3b942a66](https://www.linkedin.com/in/maryam-iqbal-3b942a66)
- Google Scholar: <https://scholar.google.com/citations?user=-4togNoAAAAJ&hl=en>



# POLYNOMIAL LINEAR REGRESSION

# CONTENTS

Why polynomial linear regression?

Mathematical Model

Several important conditions

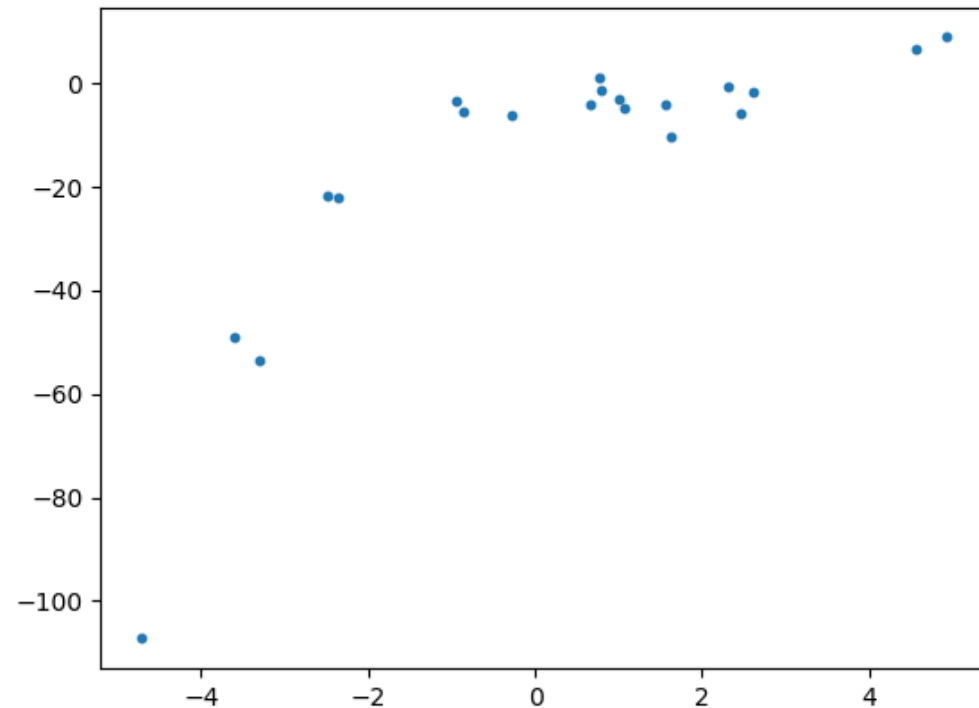
Comparison of model estimation with varying order of degree

Python implementation

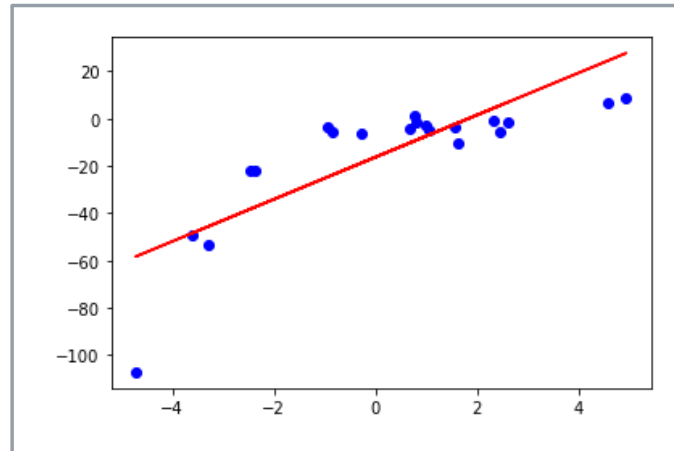
Another Example: Polynomial regression with more than one independent variable

## WHY POLYNOMIAL LINEAR REGRESSION?

To understand the need for polynomial regression, let's consider some random dataset first.



# WHY POLYNOMIAL LINEAR REGRESSION?



RMSE: 19.92996711710815  
R<sup>2</sup>: 0.6351411995779823  
Durbin-Watson: [0.73138867]

- The plot of the best fit line is
- We can see that the straight line is unable to capture the patterns in the data.
- This is an example of **under-fitting**

## WHY POLYNOMIAL LINEAR REGRESSION?



To overcome under-fitting, we need to increase the complexity of the model.



To generate a higher order equation, we can add powers of the original features as new features. The linear model,



Sometimes the polynomial regression works better e.g. predicting the spread of pandemics or diseases etc.

---

# MATHEMATICAL MODEL

Simple  
Linear  
Regression

$$y = b_0 + b_1x_1$$

Multiple  
Linear  
Regression

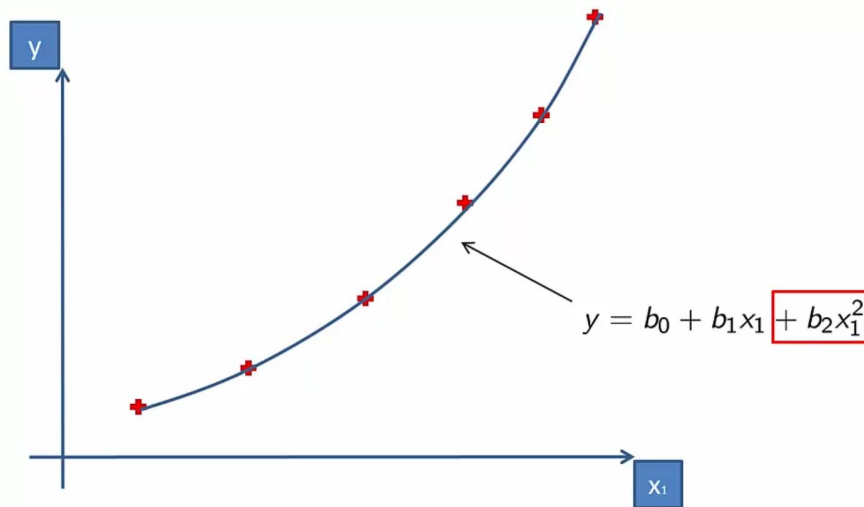
$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Polynomial  
Linear  
Regression

$$y = b_0 + b_1x_1 + b_2x_1^2 + \dots + b_nx_1^n$$

---





# POLYNOMIAL REGRESSION

Polynomial  
Linear  
Regression

$$y = b_0 + b_1x_1 + b_2x_1^2 + \dots + b_nx_1^n$$

- Polynomial models are useful in situations where the analyst knows that curvilinear effects are present in the true response function.
- Polynomial models are also useful as approximating functions to unknown and possibly very complex nonlinear relationships.
- Notice that there is only one variable  $x_1$  and its powers  $x_1^n$ .
- Although there can be more than one variable.
- If there are more than one variable then the total number of combinations for degree=N will be  $2^N - 1$
- If you include (N-1) regressors based on X, you will perfectly fit the data.

Several important conditions:

### Order of the model:

- The order ( $k$ ) should be as low as possible.
- The high-order polynomials ( $k > 2$ ) should be avoided unless they can be justified.
- In an extreme case it is always possible to pass a polynomial of order  $n-1$  through  $n$  point so that a polynomial of sufficiently high degree can always be found that provides a “good” fit to the data.

### Model Building Strategy:

- Various strategies for choosing the order of an approximating polynomial have been suggested.
- Three methods:
  - **forward selection**
  - **backward elimination.**
  - **Forward-backward elimination**

# COMPARISON OF MODEL ESTIMATION WITH VARYING ORDER OF DEGREE

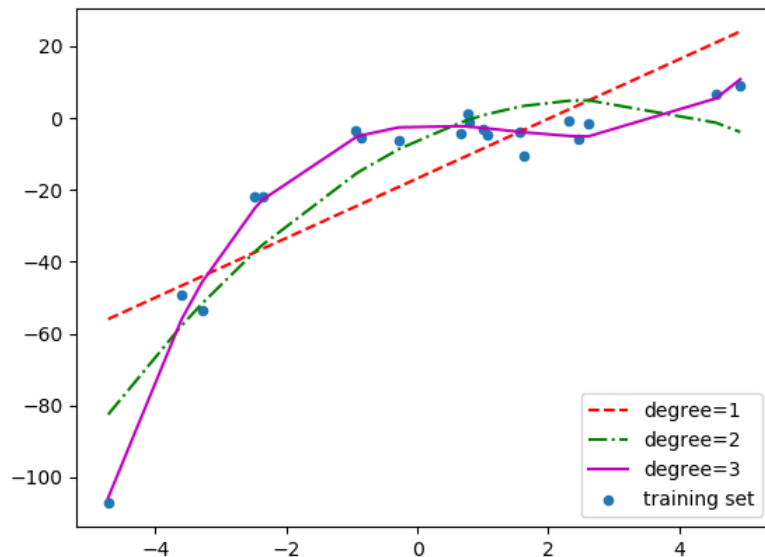
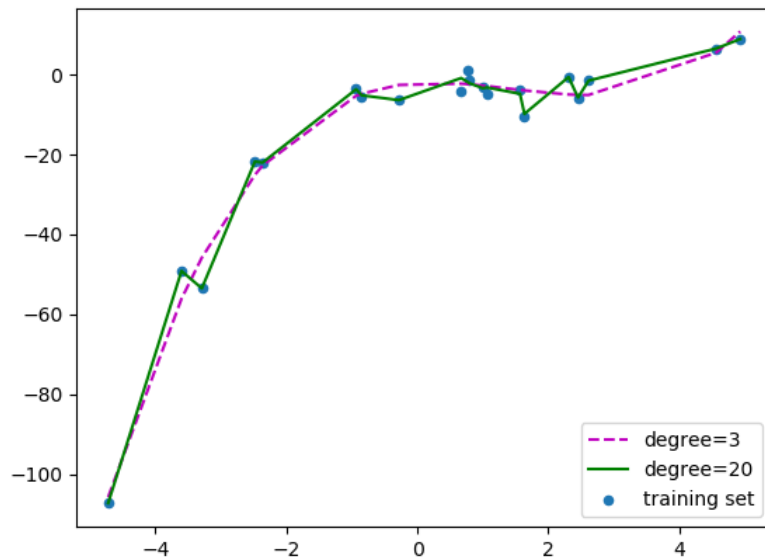
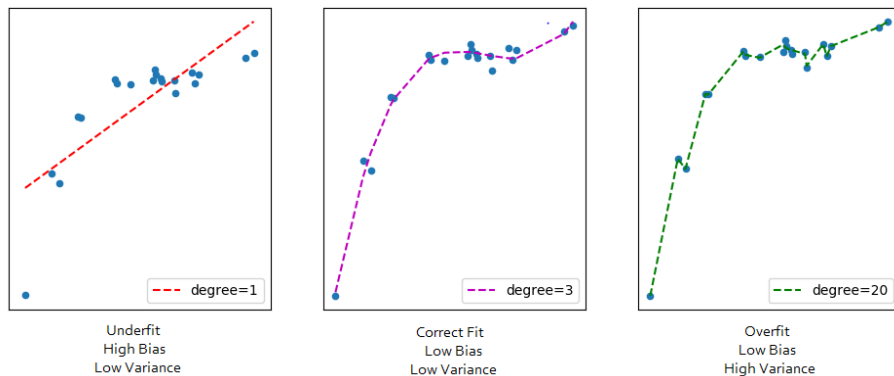


Figure shows a comparison of fitting linear, quadratic and cubic curves on the dataset

# COMPARISON OF MODEL ESTIMATION WITH VARYING ORDER OF DEGREE



- If we further increase the degree to 20, we can see that the curve passes through more data points. Figure shows a comparison of curves for degree 3 and 20.



- From the below picture we can observe that
  - As the model complexity increases, the bias decreases and the variance increases and vice-versa.
  - Ideally, a machine learning model should have **low variance and low bias**.
  - But practically it's impossible to have both.
  - Therefore to achieve a good model that performs well both on the train and unseen data, a **trade-off** is made.

Polynomial  
Linear  
Regression

$$y = b_0 + b_1x_1 + b_2x_1^2 + \dots + b_nx_1^n$$

- When we are talking about linear and non-linear regression models
- We are not talking about the relationship between y and  $x_n$ 's
- But we are talking about the relationship between y and  $b_n$ 's , which is linear as there are no  $b^2$  or  $b^3$  here

# PYTHON IMPLEMENTATION





# DEFINE THE PROBLEM



We are an HR department and we want to hire someone

We found someone who is a great fit for this job

We want to offer a position to this person in our company

Let us assume he says yes and his requested salary is \$160,000 per year, as he is very advanced in his career.

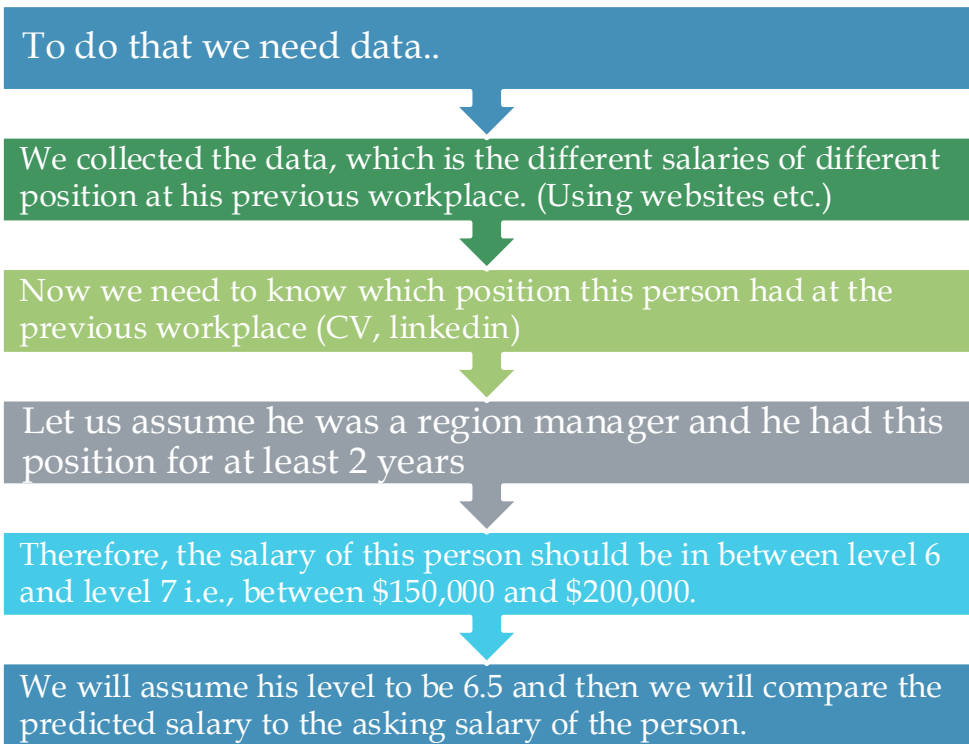
We asked why are you asking this number

He replies that is what I earned at my position in the last company I worked at.

Is that a bluff or truth? We are going to figure this out using our polynomial regression model and estimate his salary at his previous company



| Index | Position          | Level | Salary  |
|-------|-------------------|-------|---------|
| 0     | Business Analyst  | 1     | 45000   |
| 1     | Junior Consultant | 2     | 50000   |
| 2     | Senior Consultant | 3     | 60000   |
| 3     | Manager           | 4     | 80000   |
| 4     | Country Manager   | 5     | 110000  |
| 5     | Region Manager    | 6     | 150000  |
| 6     | Partner           | 7     | 200000  |
| 7     | Senior Partner    | 8     | 300000  |
| 8     | C-level           | 9     | 500000  |
| 9     | CEO               | 10    | 1000000 |



## STEPS FOR PYTHON IMPLEMENTATION

Importing libraries and dataset

Build the linear regression model

Build the polynomial regression model

Train the models

Visualize the models

Test the models (predicting the new results)

Evaluate the models

# IMPORTING THE LIBRARIES AND DATASET

```
# importing the libraries  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
  
# import the dataset  
dataset = pd.read_csv('Position_Salaries.csv')
```

- A simple dataset
- Two independent variables
  - Position
  - Level
- Dependent variable
  - Salary
- We do not need to take care of missing data
- We do not need to split the dataset into training and test set here

- Since the 'position' column and 'level' column are redundant because we can have the level column by just applying label encoder to the 'position' column so, we have left out the 'position' column
- We have used index values[:,1:-1] i.e., all columns except the first and last one

```
X = dataset.iloc[:,1:-1].values  
y = dataset.iloc[:, -1].values
```

# BUILD THE LINEAR REGRESSION MODEL

- We are building the simple linear regression model, since there is only one independent variable.

```
Slope: [80878.78787879]  
intercept: -195333.3333333332
```

```
# training the linear regression model  
from sklearn.linear_model import LinearRegression  
lin_reg = LinearRegression()  
lin_reg.fit(X,y)  
  
print('Slope: ', lin_reg.coef_)  
print('intercept: ', lin_reg.intercept_)
```

# BUILDING A POLYNOMIAL LINEAR REGRESSION MODEL

- We are going to build a multiple linear regression model with independent variables as  $x_1, x_1^2$  etc.
- Polynomial regression model is a non-linear model since, it can learn non-linear relationships, but it is called linear because of the linear combination of the variables involved

Multiple  
Linear  
Regression

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Polynomial  
Linear  
Regression

$$y = b_0 + b_1x_1 + b_2x_1^2 + \dots + b_nx_1^n$$

```
# training the polynomial linear regression model
from sklearn.preprocessing import PolynomialFeatures
poly_reg = PolynomialFeatures(degree=2)
X_poly = poly_reg.fit_transform(X)
lin_reg_2 = LinearRegression()
lin_reg_2.fit(X_poly,y)
```

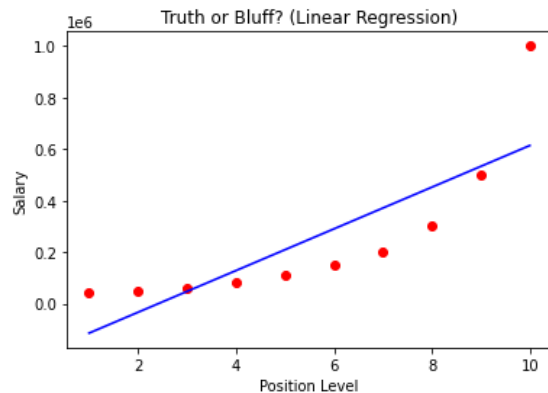
|   | 0 | 1  | 2   |
|---|---|----|-----|
| 0 | 1 | 1  | 1   |
| 1 | 1 | 2  | 4   |
| 2 | 1 | 3  | 9   |
| 3 | 1 | 4  | 16  |
| 4 | 1 | 5  | 25  |
| 5 | 1 | 6  | 36  |
| 6 | 1 | 7  | 49  |
| 7 | 1 | 8  | 64  |
| 8 | 1 | 9  | 81  |
| 9 | 1 | 10 | 100 |

- Congratulations !!!
- You have built your first polynomial linear regression model



# VISUALIZING THE DATA (LINEAR REGRESSION)

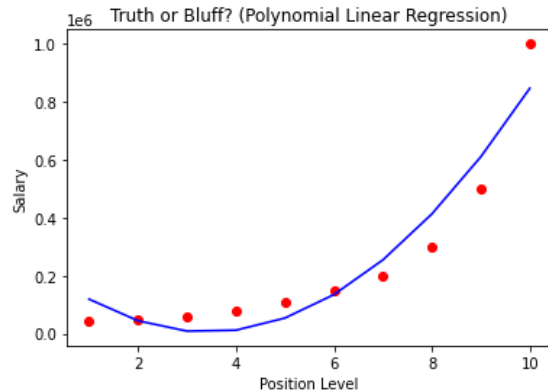
```
#Visualizing the data
# Linear Regression
plt.scatter(X, y, color='red')
plt.plot(X, lin_reg.predict(X), color='blue')
plt.title('Truth or Bluff? (Linear Regression)')
plt.xlabel('Position Level')
plt.ylabel('Salary')
plt.show()
```



- Red dots show the original data for position level vs salary
- The blue line shows the trained linear regression model
- We can see the linear regression model is not well adapted to this dataset
- The model is only valid for two values i.e., between level 2-level4 and level8-level10
- Otherwise for most of the other inputs we would have offered way more salary than others are giving.

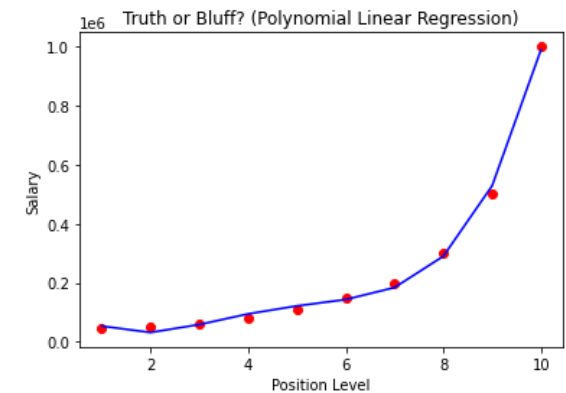
## VISUALIZING THE DATA (POLYNOMIAL LINEAR REGRESSION)

```
# polynomial Linear Regression
plt.scatter(X, y, color='red')
plt.plot(X, lin_reg_2.predict(X_poly), color='blue')
plt.title('Truth or Bluff? (Linear Regression)')
plt.xlabel('Position Level')
plt.ylabel('Salary')
plt.show()
```



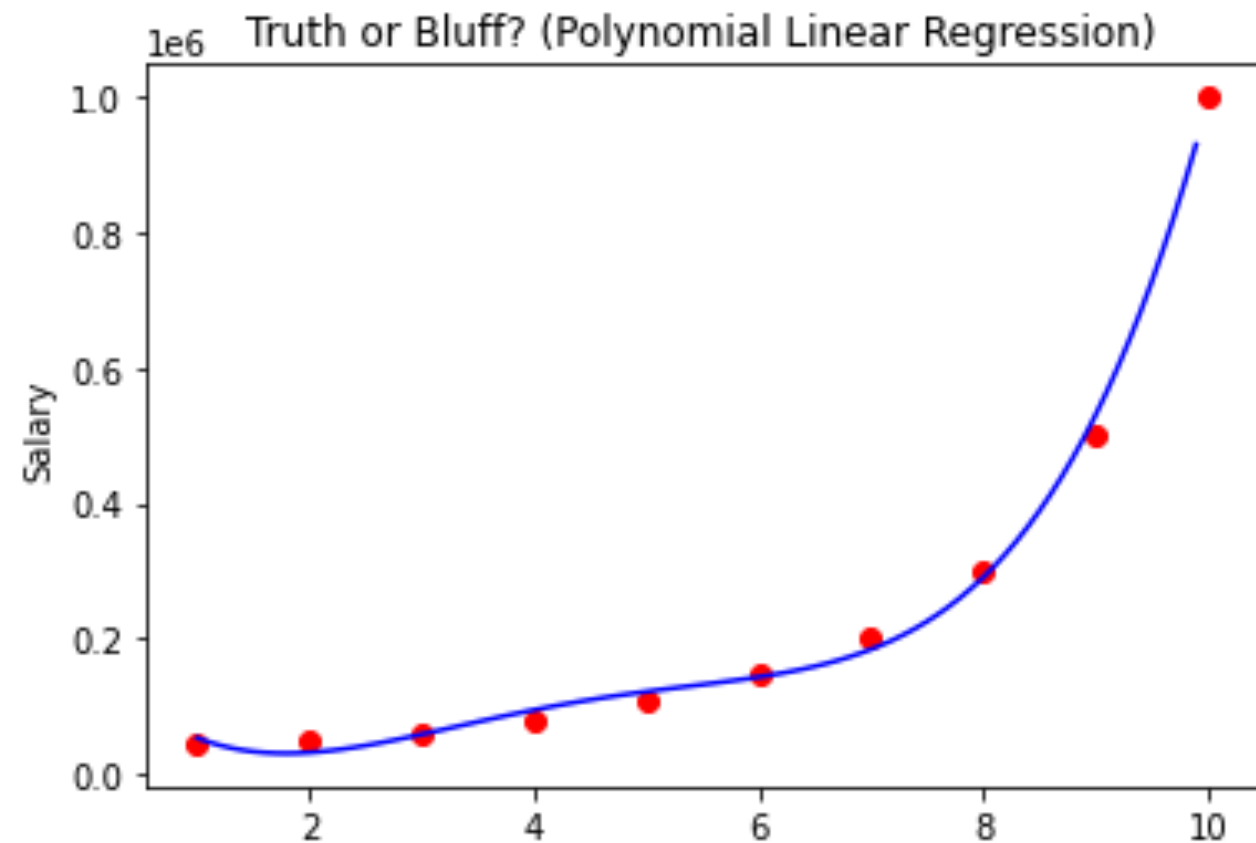
- The Polynomial Linear regression curve is better than the Linear regression model
- The predictions with Polynomial Linear regression come more closer to the real values in comparison to the Linear regression model.

```
# polynomial Linear Regression
#plt.scatter(X, y, color='red')
#plt.plot(X, lin_reg_2.predict(X_poly), color='blue')
#plt.title('Truth or Bluff? (Polynomial Linear Regression)')
#plt.xlabel('Position Level')
#plt.ylabel('Salary')
#plt.show()
```



# RESULTS WITH DEGREE=4

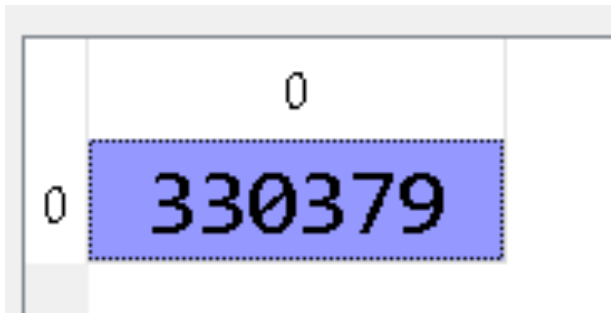
OVERFITTED MODEL



```
x_grid = np.arange(min(X),max(X),0.1)
x_grid = x_grid.reshape(len(x_grid),1)
plt.scatter(X, y, color='red')
plt.plot(x_grid, lin_reg_2.predict(poly_reg.fit_transform(x_grid)), color='blue')
plt.title('Truth or Bluff? (Polynomial Linear Regression)')
plt.xlabel('Position Level')
plt.ylabel('Salary')
plt.show()
```

- Better presented curve
- Point density for plotting here is 0.1

## PREDICTING THE NEW RESULT (LINEAR REGRESSION)



```
# predicting the output  
# linear regression  
y_pred_lin = lin_reg.predict([[6.5]])
```

- In `[[6.5]]`, the first bracket is for the rows and the second bracket is for the columns
- For example `[[2,3],[4,6]]`, we have two rows and two columns and so on
- The results from the Linear Regression Model do not seem correct as the predicted salary is too much higher than the requested salary.

## PREDICTING THE NEW RESULT (POLYNOMIAL LINEAR REGRESSION)

```
#Polynomial Linera regression  
#y_pred_lin_2 = lin_reg_2.predict([[1,6.5,6.5**2,6.5**3,6.5**4]])  
y_pred_lin_2 = lin_reg_2.predict(poly_reg.fit_transform([[6.5]]))
```



The polynomial linear regression shows better results.



The predicted salary and the requested salary are much closer together.



So, Congratulations you have a great fit for the job.

|   |        |
|---|--------|
|   | 0      |
| 0 | 158862 |

RMSE: 489532.2442718336

$R^2$ : 1.0

Durbin-Watson: 0.8618136326352376

```
#Evaluating the models
# Liner regression
residuals = abs(y)-abs(lin_reg.predict(X))
ss = (residuals**2)
ss = ss.sum()
import math
RMSE = math.sqrt(ss)
print('RMSE: ', RMSE)
linear_r2 = lin_reg.score(X, lin_reg.predict(X))
print('R^2: {}'.format(linear_r2))

# durbin_watson
from statsmodels.stats.stattools import durbin_watson
durbinWatson = durbin_watson(residuals)
print('Durbin-Watson:', durbinWatson)
```

# EVALUATING THE MODEL

## LINEAR REGRESSION

# EVALUATING THE MODEL

```
# Polynomial Liner regression
```

```
residuals = abs(y)-abs(lin_reg_2.predict(poly_reg.fit_transform(X)))
```

```
ss = (residuals**2)
```

```
ss = ss.sum()
```

```
import math
```

```
RMSE = math.sqrt(ss)
```

```
print('RMSE: ', RMSE)
```

```
linear_r2 = lin_reg.score(X, lin_reg_2.predict(poly_reg.fit_transform(X)))
```

```
print('R^2: {0}'.format(linear_r2))
```

```
# durbin_watson
```

```
from statsmodels.stats.stattools import durbin_watson
```

```
durbinWatson = durbin_watson(residuals)
```

```
print('Durbin-Watson:', durbinWatson)
```

RMSE: 45863.25575488709

R^2: 0.6707904607415657

Durbin-Watson: 2.281437775413533



# QUESTIONS

